

Woxsen University

School of Technology



A

PROJECT REPORT

on

Internship Application and Progress Monitoring System

Submitted in partial fulfillment of the requirements for the degree of

B. Tech. in Artificial Intelligence and Machine Learning

Submitted by:

Martala Harsha Vardhan Reddy 25WU0102152

Under the guidance of: Dr. Kiran Mayee, Department of Computer Science and
Engineering

1.ABSTRACT

In most academic institutions, handling student internships is still treated as an administrative afterthought, almost always run through shared spreadsheets, ad-hoc Google Forms, and endless email chains. While this works fine when a department only has ten or fifteen students, it completely breaks down during mass internship drives involving hundreds of candidates across multiple specializations. Spreadsheets lack referential integrity, meaning rows get deleted accidentally, duplicate student profiles pile up, applications point to job openings that no longer exist, and weekly progress logs get detached from their parent records. Worse, these shared files offer no realistic security boundary—anyone with a shared viewing link can mistakenly overwrite someone's grade, edit another candidate's CGPA, or alter mentor feedback notes.

To tackle these practical coordination problems, I developed the **Internship Application and Progress Monitoring System** as a complete, three-tier database web application. The core goal of the project was to move away from unorganized flat files and build a clean, reliable relational backend that reflects how real-world academic placements actually function. The system uses **MySQL Community Server** as the persistent relational engine, an asynchronous **Node.js with Express.js** server as the middle-tier API and security layer, and a custom, responsive single-page web dashboard that runs directly in the browser.

The database schema (student_intern) is structured around seven entities normalized up to Third Normal Form (3NF): Student, Company, Faculty_Mentor, Internship_Posting, Application, Internship_Record, and Progress_Report. By strictly normalizing the tables, the schema avoids insertion, update, and deletion anomalies. When a corporate partner changes their contact details or office location, that change happens in exactly one place. Foreign keys configured with cascading delete rules ensure that when records or applications are legitimately removed, all dependent milestone logs and tracking references are cleaned up automatically without leaving orphaned data behind.

A central engineering priority of the application was enforcing practical Role-Based Access Control (RBAC). Rather than trusting client-side JavaScript alone to hide buttons, the system creates two completely separate operating experiences:

- **Viewer Mode:** The default state when opening the dashboard. General visitors, students, and recruiters can freely browse company partner directories, inspect active

job vacancies, and check progressive milestone summaries. To keep database performance high and prevent visitors from running heavy multi-table joins, the backend serves this public data through three dedicated MySQL views: `view_active_postings`, `view_student_records`, and `view_progress_reports`. All data entry forms and delete actions are kept completely hidden.

- **Owner Mode:** Intended strictly for authorized university placement administrators. Access is unlocked through a native in-page modal prompt using a protected passkey handshake (Harsha@2026). Once authenticated, the browser session enables full write permissions, dynamically displaying insert forms and deletion controls for all seven database tables. On the backend, custom Express middleware (`requireOwner`) inspects an authorization header on every single POST and DELETE request, instantly returning an HTTP 403 Forbidden status code if an unauthenticated user tries to modify the database through terminal calls or external API clients.

The entire system was seeded and validated using an authentic benchmark dataset containing thirty relational rows per table (totalling 210 connected records). A lightweight, real-time search engine was built directly into the client script, allowing users to type any keyword—such as a student roll number, company location, role title, or status keyword—and watch every table on the dashboard filter down simultaneously without triggering wasteful database queries. By combining strict relational integrity, SQL view encapsulation, robust server-side route guards, and an uncluttered dark-mode UI, the project demonstrates how traditional, error-prone academic paperwork can be replaced by an intuitive, production-ready database management system.

2. INTRODUCTION AND PROBLEM STATEMENT

2.1 Detailed Background & Institutional Context

In undergraduate engineering programs, technical internships serve as the bridge between academic coursework and real software engineering practices. As students move into their fourth and fifth semesters, they branch out into specialized domains like cloud infrastructure, systems programming, web technologies, and machine learning. To ensure these industrial training stints actually carry academic value, universities require students to complete these placements under the joint supervision of an external corporate mentor and an internal faculty guide.

On paper, this sounds straightforward. But in practice, managing an internship cycle requires constantly synchronizing four distinct stakeholders who operate with completely different priorities:

1. **The Students:** Need a clear directory of available job postings, transparent eligibility criteria (such as semester requirements and minimum academic standing), and a reliable channel to log their weekly technical deliverables without chasing faculty through corridors or email threads.
2. **Corporate Recruiters:** Need their organization profiles, branch locations, role descriptions, and stipend amounts represented accurately without having to repeatedly email updated contact sheets to academic cells.
3. **Faculty Mentors:** Are responsible for guiding anywhere from fifteen to thirty students simultaneously. They need an uncluttered view of each student's weekly progress, a historical timeline of tasks completed, and a quick way to leave actionable feedback or flag issues before final grade credits are tallied.
4. **Placement Coordinators:** Need macro-level visibility over the entire academic cohort—how many students are actively interning, which partner companies have taken students, which applications are still pending review, and which records have concluded successfully.

2.2 In-Depth Problem Statement

Despite the high stakes of placement monitoring, most engineering departments still run their entire workflow on ad-hoc collections of Google Sheets, Microsoft Excel files, shared cloud folders, and email chains. Over a full semester, this setup causes serious breakdowns across four main areas:

- **Silent Update and Deletion Anomalies:**

Spreadsheets store data in flat grids where company names, student details, and mentor assignments are typed out repeatedly across multiple columns and tabs. If a recruiter changes their talent acquisition email, that change is usually updated in one coordinator's private sheet while older sheets circulating among students keep displaying invalid addresses. Worse, if a coordinator deletes a student's row because they changed their academic section, every associated record of which company they applied to vanishes along with it, creating permanent blind spots.

- **Complete Absence of Referential Integrity:**

Flat files cannot enforce relational rules. There is nothing stopping someone from pasting a typo like P999 into an application column when that posting ID doesn't exist anywhere in the company list. Similarly, weekly milestone logs frequently become disconnected from the student who submitted them when rows are sorted alphabetically or filtered incorrectly by another user. Without foreign keys and cascading rules, data quickly becomes corrupted.

- **Zero Granular Access Control:**

Shared spreadsheets operate on an "all-or-nothing" permission model. If coordinators share a link so students can view available job openings, anyone with editing rights can accidentally overwrite another student's CGPA, modify their application status from Rejected to Accepted, or alter mentor evaluation remarks. To prevent this, coordinators often resort to making sheets read-only and handling every single update via email, which creates an unnecessary administrative bottleneck.

- **Fragmented Evaluation Timelines:**

Faculty guides do not have time to comb through messy email threads to check if an intern has logged their Week 3 or Week 4 deliverables. Because there is no single dashboard linking students directly to their weekly submissions, faculty evaluations are often delayed until the very end of the semester, turning what should be continuous mentorship into a rushed, last-minute grading exercise.

3. OBJECTIVES AND DETAILED SCOPE

3.1 Core Technical Objectives

To resolve these issues, this project was developed with five clear technical objectives:

1. **Design an Anomaly-Free Relational Schema (3NF):**

Create an organized MySQL schema where every real-world entity lives in its own dedicated table, eliminating duplicate data so that any update (like changing a mentor's email or a posting's stipend) happens in exactly one place.

2. **Enforce Referential Integrity with Cascades:**

Implement primary and foreign key constraints across all seven tables. Configure cascading rules so that if a student profile or an application is legitimately removed, dependent records and milestone logs are cleaned up automatically without leaving orphaned data behind.

3. **Abstract Relational Complexity via SQL Views:**

Build pre-joined database views (view_active_postings, view_student_records, view_progress_reports) so the frontend can retrieve aggregated reports quickly without running expensive multi-table joins on every page load.

4. **Build Real, Dual-Layer Role-Based Access Control (RBAC):**

Enforce access boundaries on both the frontend and backend. Provide a public **Viewer Mode** where visitors can search and inspect records, and an authenticated **Owner Mode** protected by server-side middleware (requireOwner) that blocks unauthorized API mutations with HTTP 403 Forbidden responses.

5. **Deliver an Intuitive, Responsive Dashboard:**

Build a single-page web interface with real-time global search, custom compact scrollbars, and full-width tables to prevent horizontal clipping, ensuring that managing 200+ database records feels effortless on standard screens.

3.2 Explicit Scope Boundaries

To keep the engineering effort focused on database principles, normalization, and secure data access, the scope was intentionally bounded:

- **Included in Scope:** Academic student profiles, corporate partner directories, internship vacancy listings, student application tracking, confirmed placement records, progressive weekly milestone logs, faculty review comments, and global real-time search.
- **Excluded from Scope:** PDF resume parsing, automated background payroll processing, and third-party payment gateways. These features add external complexity without providing additional value for evaluating core relational database concepts.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Developing a responsive 3-tier database web application requires selecting toolchains and hardware that handle persistent relational queries, server routing, and clean client rendering without performance bottlenecks. Rather than simply listing names, the specifications below detail the exact role and justification for every component in the system.

4.1 Software Requirements & Toolchain Justification

- **Relational Database Management System: MySQL Community Server (Version 8.0+)**
 - *Role:* Acts as the persistent relational storage engine for the `student_intern` schema.
 - *Justification:* MySQL was chosen for its mature, ACID-compliant **InnoDB engine**, which provides rock-solid support for transactional safety, physical primary/foreign key enforcement, and cascading deletions (ON DELETE CASCADE). Its query optimizer efficiently computes multi-table joins under the hood, making our database views (`view_active_postings`, `view_student_records`, `view_progress_reports`) fast and responsive during high-frequency read operations.
- **Backend Server Environment: Node.js (v20.x LTS)**
 - *Role:* Serves as the asynchronous runtime environment powering our application tier.
 - *Justification:* Node.js uses an event-driven, non-blocking I/O model that is well-suited for database web backends. Instead of dedicating a heavy OS-level thread to each client connection, it processes database queries asynchronously through the V8 JavaScript engine, preventing server lockups during peak dashboard usage.
- **API & Web Application Framework: Express.js (v4.x)**
 - *Role:* Manages RESTful routing, parses incoming JSON payloads, serves static frontend assets, and executes access-control middleware.
 - *Justification:* Express provides a lightweight, unopinionated routing layer. It makes writing modular API endpoints straightforward and allows us to insert the custom `requireOwner` security middleware directly in front of all mutating routes (POST, DELETE), keeping write operations completely secure.

- **Database Driver / Connection Pooling: mysql2/promise**
 - *Role:* Manages low-level TCP/IP socket connections between Node.js and the local MySQL server on port 3306.
 - *Justification:* Opening and tearing down a brand-new database handshake for every single API request adds noticeable latency. By configuring a **connection pool** with a limit of 10 concurrent reusable connections, mysql2/promise eliminates handshake overhead, automatically handles connection recycling, and allows us to write modern async/await syntax for clean asynchronous database queries.

- **Frontend Technology Stack: HTML5, Modern CSS, Vanilla JavaScript (ES6+)**
 - *Role:* Powers the interactive single-page dashboard loaded by the user's browser.
 - *Justification:* Heavy single-page frameworks (such as React, Angular, or Vue) introduce complex compilation toolchains, state overhead, and virtual DOM diffing that are unnecessary for a clean administrative dashboard. Relying on standard ES6+ JavaScript (fetch(), Promise.all(), DOM event listeners) ensures instant page execution without any build steps. The native HTML5 <dialog> element provides a built-in, unblockable modal prompt for owner authentication, while modern CSS Grid and Flexbox handle responsive dashboard layouts and custom 6px scrollbars cleanly.

- **Web Browser Client: Google Chrome (v120+) / Mozilla Firefox (v120+)**
 - *Role:* Serves as the primary client execution and debugging environment.

- *Justification:* While lightweight embedded webviews (like the VS Code Simple Browser preview) work for quick file checks, full desktop browsers provide complete DevTools panels (F12). This allows direct inspection of network payload headers (x-owner-key), browser storage verification (sessionStorage), and cache-free reloading (Ctrl + F5) during development.
- **Integrated Development Environment (IDE): Visual Studio Code**
 - *Role:* Primary code editor, workspace manager, and integrated terminal environment.
 - *Justification:* Offers integrated multi-tab PowerShell/Bash terminals, syntax highlighting for SQL and JavaScript, and extensions for running MySQL queries directly inside the workspace without switching between external desktop clients.

4.2 Hardware Specifications & Operational Environment

The hardware configuration used to develop, test, and host the system locally is outlined below:

- **Processor (CPU):**
 - *Specification:* 64-bit Multi-Core Processor (Intel Core i5 11th Gen / AMD Ryzen 5 4000 series or higher, minimum 4 physical cores, base clock $\geq$ 2.4 GHz).
 - *Operational Need:* Necessary to run both the MySQL database daemon (mysqld.exe) and the Node.js runtime concurrently alongside client browser rendering without CPU throttling or thread contention.
- **System Memory (RAM):**
 - *Specification:* 8 GB DDR4 minimum (16 GB recommended).
 - *Operational Need:* MySQL InnoDB assigns a dedicated buffer pool in memory to cache indexes and table data pages. Running the OS, background daemons, the Node runtime, and modern browser tabs (which consume substantial memory for DevTools) requires at least 8 GB of RAM to prevent memory paging to disk.

- **Secondary Storage (SSD):**

- *Specification:* 512 GB Solid State Drive (NVMe M.2 or SATA III) with at least 500 MB of dedicated free disk space.
- *Operational Need:* Fast random read/write speeds (≥ 500 MB/s) are critical for relational databases. SSD storage eliminates disk-head seek times when MySQL writes to write-ahead redo logs, ensuring queries and schema rebuilds execute in milliseconds.

- **Display Resolution & Screen Workspace:**

- *Specification:* Full HD (1920×1080 resolution, 16:9 aspect ratio).
- *Operational Need:* Essential for the visual presentation of our multi-table dashboard. Because the system renders 6-to-7 column relational tables (such as Active Postings and Student Records), a 1080p display ensures that tables breathe naturally across the full width without horizontal text clipping or awkward overlapping.

- **Network Interface:**

- *Specification:* Standard Local Loopback Adapter (localhost / 127.0.0.1) over TCP/IP.
- *Operational Need:* Handles internal socket communication between the browser front-end (port 3000), the Node Express API server (port 3000), and the MySQL database daemon (port 3306) with near-zero latency.

5.ER Diagram

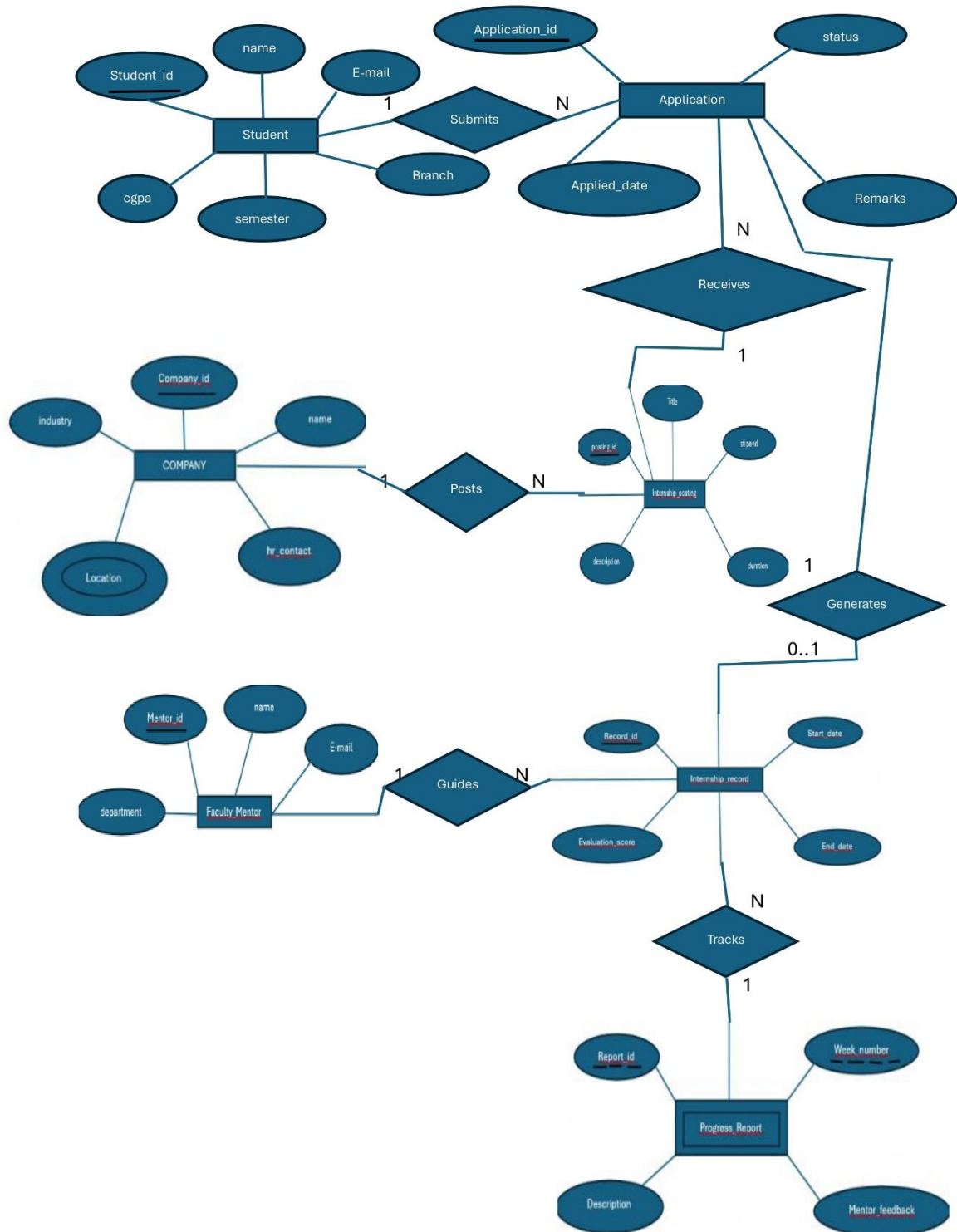


Fig 5: ER Diagram for the Internship Application and Progress Monitoring System.

6. RELATIONAL SCHEMA AND STEP-BY-STEP NORMALIZATION

6.1 Relational Schema Mapping

Mapping our conceptual Entity-Relationship model into a physical relational schema requires converting each entity and its cardinalities into distinct tables. Primary keys are underlined, and foreign keys are denoted with an asterisk (*):

- **Student** (student_id, name, email, program, semester, cgpa)
- **Company** (company_id, name, industry, location, contact)
- **Faculty_Mentor** (faculty_id, name, department, email)
- **Internship_Posting** (posting_id, company_id*, title, description, duration, stipend)
- **Application** (application_id, student_id*, posting_id*, status, applied_date)
- **Internship_Record** (record_id, application_id*, faculty_id*, start_date, end_date, final_status)
- **Progress_Report** (record_id*, week_num, description, mentor_feedback)

6.2 Step-by-Step Normalization Analysis

Database normalization is the deliberate process of organizing fields and tables to eliminate data redundancy and guard against insertion, update, and deletion anomalies. Below is the step-by-step transition from an unnormalized state up to Third Normal Form (3NF):

1. First Normal Form (1NF)

- **The Practical Problem:** In typical spreadsheet tracking, departmental coordinators frequently stuff multi-valued entries into single cells to save visual space. For instance, a single column titled Weekly Updates might contain comma-separated text entries like:

"Week 1: Local environment setup, Week 2: REST routing, Week 3: MySQL testing".

This makes it impossible for standard SQL SELECT or UPDATE queries to modify, filter, or evaluate an individual week's progress without using string-splitting functions.

- **The Normalization Fix:** Every attribute in our relational schema is restricted to storing purely atomic (indivisible) values. The repeating weekly reports were extracted into their own dedicated relation, Progress_Report. Furthermore, each row in every table is guaranteed a unique identity through explicitly defined primary keys (student_id, company_id, posting_id, etc.), fully satisfying 1NF.

2. Second Normal Form (2NF)

- **The Practical Problem:** 2NF requires the schema to be in 1NF and have zero partial functional dependencies—meaning no non-key attribute can be dependent on merely

a subset of a composite primary key. In our schema, only Progress_Report uses a composite primary key: (record_id, week_num). If we had kept attributes like student_name, program, or company_name inside Progress_Report, those fields would depend solely on record_id and not on week_num, which would introduce partial dependencies and waste storage on every weekly log submission.

- **The Normalization Fix:** All parent entity fields were removed from Progress_Report. The only non-key fields preserved in the table are description (the work executed that week) and mentor_feedback (the faculty guide's review remarks). Both attributes are functionally dependent on the entire composite primary key: you need to know both *which* internship placement is being evaluated and *which* specific week number is being reviewed. All other relations use single-attribute primary keys, trivially satisfying 2NF.

3. Third Normal Form (3NF)

- **The Practical Problem:** 3NF requires the schema to be in 2NF and have zero transitive dependencies—meaning no non-prime attribute may depend on another non-prime attribute (Non-Key → Non-Key). Consider Internship_Record: if we had stored company_location directly inside Internship_Record, that value would depend on company_id, which in turn depends on posting_id, which depends on application_id, which finally depends on record_id:

record_id → application_id → posting_id → company_id → company_location

If a corporate partner moved its office branch from Hyderabad to Bangalore, we would have had to locate and update every single internship record linked to that firm, running the risk of inconsistent data if even a single row was skipped.

- **The Normalization Fix:** All transitive dependencies were removed. Internship_Record stores only its direct transactional fields (start_date, end_date, final_status) alongside clean foreign key pointers to Application and Faculty_Mentor. Whenever company names, offices, or student emails are required, the application retrieves them dynamically through relational joins or pre-joined SQL views. Because every non-key attribute depends directly, exclusively, and non-transitively on the primary key, the entire database satisfies 3NF.

7. DATA DICTIONARY

The tables below provide the physical schema data dictionary for all seven tables in the student_intern database, aligned directly with the active MySQL table descriptions (DESC output):

Table 1: application

Column Name	Data Type	Nullable	Key	Default	Constraints & Technical Purpose
application_id	VARCHAR(15)	NO	PK	NULL	Primary Key; unique application tracking ID (e.g., A301)
status	VARCHAR(20)	YES	-	'Pending'	Application review status (Pending, Accepted, Rejected)
applied_date	DATE	NO	-	NULL	Formal date of application submission
resume_url	VARCHAR(255)	YES	-	NULL	Digital storage hyperlink to candidate resume document
student_id	VARCHAR(15)	NO	FK (MUL)	NULL	References student(student_id)
posting_id	VARCHAR(15)	NO	FK (MUL)	NULL	References internship_posting(posting_id)

Table 2: company

Column Name	Data Type	Nullable	Key	Default	Constraints & Technical Purpose
company_id	VARCHAR(15)	NO	PK	NULL	Primary Key; corporate partner code (e.g., C201)
name	VARCHAR(100)	NO	-	NULL	Full registered organization name
industry	VARCHAR(50)	YES	-	NULL	Corporate domain vertical (e.g., Cloud & AI)
location	VARCHAR(100)	YES	-	NULL	Office branch city (e.g., Hyderabad, Bangalore)
contact	VARCHAR(50)	YES	-	NULL	Recruitment talent acquisition contact email

Table 3: faculty_mentor

Column Name	Data Type	Nullable	Key	Default	Constraints & Technical Purpose
faculty_id	VARCHAR(15)	NO	PK	NULL	Primary Key; unique faculty staff code (e.g., F501)
name	VARCHAR(100)	NO	-	NULL	Full academic name and title
department	VARCHAR(50)	YES	-	NULL	Academic department (e.g., Computer Science & Eng)
email	VARCHAR(100)	NO	Unique (UNI)	NULL	Institutional university faculty email address

Table 4: internship_posting

Data Type	Nullable	Key	Default	Constraints & Technical Purpose
VARCHAR(15)	NO	PK	NULL	Primary Key; vacancy identifier (e.g., P401)
VARCHAR(100)	NO	-	NULL	Position title (e.g., Software Engineering Intern)
TEXT	YES	-	NULL	Detailed role requirements and deliverables
VARCHAR(30)	YES	-	NULL	Internship placement duration (e.g., 12 Weeks)
DECIMAL(10,2)	YES	-	NULL	Monthly monetary compensation in INR
VARCHAR(15)	YES	FK (MUL)	NULL	References company(company_id)

Table 5: internship_record

Column Name	Data Type	Nullable	Key	Default	Constraints & Technical Purpose
record_id	VARCHAR(15)	NO	PK	NULL	Primary Key; confirmed placement record (e.g., R601)
start_date	DATE	NO	-	NULL	Formal internship start date
end_date	DATE	YES	-	NULL	Anticipated or actual completion date
final_status	VARCHAR(20)	YES	-	'Ongoing'	Placement progress state (Ongoing, Completed)
evaluation_score	DECIMAL(4,2)	YES	-	NULL	Performance evaluation marks / score out of 100.00
application_id	VARCHAR(15)	NO	Unique (UNI)	NULL	References application(application_id) (1: 1 link)
faculty_id	VARCHAR(15)	YES	FK (MUL)	NULL	References faculty_mentor(faculty_id)

Table 6: progress_report

Column Name	Data Type	Nullable	Key	Default	Constraints & Technical Purpose
record_id	VARCHAR(15)	NO	Composite PK	NULL	References internship_record(record_id)
week_num	INT	NO	Composite PK	NULL	Milestone week number index ($1 \leq \text{week} \leq 52$)
description	TEXT	NO	-	NULL	Summary of technical work and deliverables produced
mentor_feedback	TEXT	YES	-	NULL	Faculty review notes and qualitative evaluation

Table 7: student

Column Name	Data Type	Nullable	Key	Default	Constraints & Technical Purpose
student_id	VARCHAR(15)	NO	PK	NULL	Primary Key; student roll number (e.g., S101)
name	VARCHAR(100)	NO	-	NULL	Full student candidate name

Column Name	Data Type	Nullable	Key	Default	Constraints & Technical Purpose
email	VARCHAR(100)	NO	Unique (UNI)	NULL	Institutional domain email address
program	VARCHAR(50)	YES	-	NULL	Academic degree track (e.g., B.Tech CSE)
semester	INT	YES	-	NULL	Current active semester ($1 \leq \text{sem} \leq 8$)
cgpa	DECIMAL(3,2)	YES	-	NULL	Cumulative Grade Point Average ($0.00 \leq \text{cgpa} \leq 10.00$)

8. SQL COMMANDS USED (DDL, DML) WITH SAMPLE OUTPUTS

8.1 Data Definition Language (DDL)

The physical database tables were generated using MySQL InnoDB directives to guarantee referential integrity and support cascade operations:

```
CREATE DATABASE student_intern;
```

```
USE student_intern;
```

-- 1. Student Master Table

```
CREATE TABLE student (  
    student_id VARCHAR(15) PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    program VARCHAR(50),  
    semester INT,  
    cgpa DECIMAL(3,2) CHECK (cgpa >= 0.00 AND cgpa <= 10.00)  
) ENGINE=InnoDB;
```

-- 2. Corporate Partners Table

```
CREATE TABLE company (  
    company_id VARCHAR(15) PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    industry VARCHAR(50),  
    location VARCHAR(100),  
    contact VARCHAR(50)  
) ENGINE=InnoDB;
```

-- 3. Faculty Mentors Table

```
CREATE TABLE faculty_mentor (  
    faculty_id VARCHAR(15) PRIMARY KEY,
```

```
name VARCHAR(100) NOT NULL,  
department VARCHAR(50),  
email VARCHAR(100) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

-- 4. Internship Postings Table

```
CREATE TABLE internship_posting (  
posting_id VARCHAR(15) PRIMARY KEY,  
title VARCHAR(100) NOT NULL,  
description TEXT,  
duration VARCHAR(30),  
stipend DECIMAL(10,2),  
company_id VARCHAR(15),  
FOREIGN KEY (company_id) REFERENCES company(company_id) ON DELETE  
CASCADE  
) ENGINE=InnoDB;
```

-- 5. Student Applications Table

```
CREATE TABLE application (  
application_id VARCHAR(15) PRIMARY KEY,  
status VARCHAR(20) DEFAULT 'Pending',  
applied_date DATE NOT NULL,  
resume_url VARCHAR(255),  
student_id VARCHAR(15) NOT NULL,  
posting_id VARCHAR(15) NOT NULL,  
FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE  
CASCADE,  
FOREIGN KEY (posting_id) REFERENCES internship_posting(posting_id) ON DELETE  
CASCADE  
) ENGINE=InnoDB;
```

-- 6. Monitored Internship Records Table (1:1 with Application)

```
CREATE TABLE internship_record (  
    record_id VARCHAR(15) PRIMARY KEY,  
    start_date DATE NOT NULL,  
    end_date DATE,  
    final_status VARCHAR(20) DEFAULT 'Ongoing',  
    evaluation_score DECIMAL(4,2),  
    application_id VARCHAR(15) NOT NULL UNIQUE,  
    faculty_id VARCHAR(15),  
    FOREIGN KEY (application_id) REFERENCES application(application_id) ON DELETE  
    CASCADE,  
    FOREIGN KEY (faculty_id) REFERENCES faculty_mentor(faculty_id) ON DELETE  
    CASCADE  
) ENGINE=InnoDB;
```

-- 7. Weekly Progress Milestone Log Table

```
CREATE TABLE progress_report (  
    record_id VARCHAR(15) NOT NULL,  
    week_num INT NOT NULL,  
    description TEXT NOT NULL,  
    mentor_feedback TEXT,  
    PRIMARY KEY (record_id, week_num),  
    FOREIGN KEY (record_id) REFERENCES internship_record(record_id) ON DELETE  
    CASCADE  
) ENGINE=InnoDB;
```

8.2 Data Manipulation Language (DML) Commands & Sample Queries

DML statements execute core operational workflows: registering students, creating company job postings, tracking application statuses, and maintaining weekly reviews:

-- Inserting a new student profile

```
INSERT INTO student (student_id, name, email, program, semester, cgpa)
VALUES ('S101', 'Martala Harsha Vardhan Reddy', 'harsha@univ.edu', 'B.Tech CSE', 3,
8.85);
```

-- Inserting an active internship opening

```
INSERT INTO internship_posting (posting_id, title, description, duration, stipend,
company_id)
VALUES ('P401', 'Software Engineering Intern', 'Full stack development with Node.js and
React.', '12 Weeks', 50000.00, 'C201');
```

-- Submitting a student application

```
INSERT INTO application (application_id, status, applied_date, resume_url, student_id,
posting_id)
VALUES ('A301', 'Accepted', '2026-05-10', NULL, 'S101', 'P401');
```

-- Logging an ongoing internship placement record

```
INSERT INTO internship_record (record_id, start_date, end_date, final_status,
evaluation_score, application_id, faculty_id)
VALUES ('R601', '2026-06-01', '2026-08-24', 'Ongoing', NULL, 'A301', 'F501');
```

-- Submitting a weekly progress report

```
INSERT INTO progress_report (record_id, week_num, description, mentor_feedback)
VALUES ('R601', 1, 'Completed enterprise onboarding, dev environment setup, and
repository access.', 'Setup verified on schedule.');
```

-- Deleting a student record (ON DELETE CASCADE cleans up dependent application & record entries)

```
DELETE FROM student WHERE student_id = 'S130';
```

8.3 Sample Query Outputs From Live Database

Output 1: SELECT * FROM student LIMIT 5;

student_id	name	email	program	semester	cgpa
S101	Martala Harsha Vardhan Reddy	harsha@univ.edu	B.Tech CSE	3	8.85
S102	Aarav Sharma	aarav.sharma@univ.edu	B.Tech CSE	3	9.12
S103	Diya Patel	diya.patel@univ.edu	B.Tech AI & DS	5	8.45
S104	Rohan Verma	rohan.verma@univ.edu	B.Tech CSE	4	7.80
S105	Ananya Iyer	ananya.iyer@univ.edu	B.Tech Cyber Security	5	9.30

5 rows in set (0.011 sec)

Output 2: SELECT * FROM company LIMIT 5;

company_id	name	industry	location	contact
C201	Google India	Cloud & AI	Hyderabad	careers-india@google.com
C202	Microsoft R&D	Enterprise Tech	Hyderabad	in-recruiting@microsoft.com
C203	Amazon Dev Center	E-Commerce & AWS	Bangalore	aws-interns@amazon.com
C204	Salesforce	CRM & Cloud	Hyderabad	talent-india@salesforce.com
C205	NVIDIA Graphics	Semiconductors & AI	Bangalore	nv-university@nvidia.com

5 rows in set (0.008 sec)

Output 3: SELECT * FROM faculty_mentor LIMIT 5;

faculty_id	name	department	email
F501	Dr. Ramesh Babu	Computer Science & Eng	ramesh.b@univ.edu
F502	Dr. Sunita Sharma	Artificial Intelligence	sunita.s@univ.edu
F503	Prof. Venkat Raman	Information Technology	venkat.r@univ.edu
F504	Dr. Shalini Gupta	Data Science	shalini.g@univ.edu
F505	Prof. Arvind Swamy	Computer Science & Eng	arvind.s@univ.edu

5 rows in set (0.007 sec)

Output 4: SELECT * FROM internship_posting LIMIT 5;

posting_id	title	description	duration	stipend	company_id
P401	Software Engineering Intern	Full stack development with Node.js and React.	12 Weeks	50000.00	C201
P402	Cloud Infrastructure Intern	Azure microservices and Kubernetes automation.	16 Weeks	48000.00	C202
P403	Data Science Intern	Customer intent analytics and recommendation systems.	12 Weeks	45000.00	C203
P404	Salesforce Developer Intern	Apex triggers and Lightning component development.	10 Weeks	40000.00	C204
P405	GPU Kernel Optimizer Intern	CUDA and C++ low-level performance profiling.	16 Weeks	55000.00	C205

5 rows in set (0.007 sec)

Output 5: SELECT * FROM application LIMIT 5;

application_id	status	applied_date	resume_url	student_id	posting_id
A301	Accepted	2026-05-10	NULL	S101	P401
A302	Accepted	2026-05-12	NULL	S102	P402
A303	Accepted	2026-05-15	NULL	S103	P403
A304	Accepted	2026-05-16	NULL	S104	P404
A305	Accepted	2026-05-18	NULL	S105	P405

5 rows in set (0.008 sec)

Output 6: SELECT * FROM internship_record LIMIT 5;

record_id	start_date	end_date	final_status	evaluation_score	application_id	faculty_id
R601	2026-06-01	2026-08-24	Ongoing	NULL	A301	F501
R602	2026-06-01	2026-09-15	Ongoing	NULL	A302	F502
R603	2026-05-15	2026-07-25	Completed	NULL	A303	F503
R604	2026-06-10	2026-08-30	Ongoing	NULL	A304	F504
R605	2026-05-20	2026-07-30	Completed	NULL	A305	F505

5 rows in set (0.007 sec)

Output 7: SELECT * FROM progress_report LIMIT 5;

record_id	week_num	description	mentor_feedback
R601	1	Completed enterprise onboarding, dev environment setup, and repository access.	Setup verified on schedule.
R601	2	Constructed Express routing layer for REST APIs and connected MySQL pool.	Clean code structure maintained.
R602	1	Configured Azure DevOps pipelines and initial resource groups.	Good documentation on deployment steps.
R602	2	Deployed sample container to Kubernetes cluster with ingress controller.	Excellent grasp of pod autoscaling.
R603	1	Conducted exploratory data analysis (EDA) on customer retention datasets.	Strong statistical summary delivered.

5 rows in set (0.008 sec)

9. QUERIES WITH OUTPUTS (INCLUDING PRESENTATION-II QUERY)

9.1 Presentation-II Assigned Query & Solution

Assigned Question:

"Retrieve all information about the faculty mentor named Venkat."

Query Explanation & Pattern Matching:

To look up faculty mentor records where the title or full name includes "Venkat" (e.g., Prof. Venkat Raman), the SQL LIKE operator is used with wildcards (%Venkat%). This performs substring pattern matching against the name column of the faculty_mentor table.

SQL Query:

```
SELECT * FROM faculty_mentor WHERE name LIKE "%Venkat%";
```

Live Terminal Output:

```
mysql> select * from faculty_mentor where name like "%Venkat%";
+-----+-----+-----+-----+
| faculty_id | name           | department           | email           |
+-----+-----+-----+-----+
| F503       | Prof. Venkat Raman | Information Technology | venkat.r@univ.edu |
+-----+-----+-----+-----+
1 row in set (0.251 sec)
```

9.2 Database Views Implementation Queries & Live Outputs

To reduce query overhead on client dashboard loads and provide clean, read-only data abstractions for public viewing, three relational views were created:

1. Active Postings View (view_active_postings)

- **Purpose:** Combines job openings with partner company names and locations so visitors can browse vacancies without accessing the underlying corporate primary keys directly.

SQL:

```
CREATE VIEW view_active_postings AS
```

```
SELECT
```

```
    p.posting_id,
```

```
    COALESCE(c.name, 'Independent Partner') AS company_name,
```

```
    c.location,
```

```
    p.title AS role_title,
```

```
    p.stipend,
```

```
    p.duration
```

FROM internship_posting p

LEFT JOIN company c ON p.company_id = c.company_id;

Sample Execution:

SELECT * FROM view_active_postings LIMIT 5;

Live Terminal Output:

posting_id	company_name	location	role_title	stipend	duration
P401	Google India	Hyderabad	Software Engineering Intern	50000.00	12 Weeks
P402	Microsoft R&D	Hyderabad	Cloud Infrastructure Intern	48000.00	16 Weeks
P403	Amazon Dev Center	Bangalore	Data Science Intern	45000.00	12 Weeks
P404	Salesforce	Hyderabad	Salesforce Developer Intern	40000.00	10 Weeks
P405	NVIDIA Graphics	Bangalore	GPU Kernel Optimizer Intern	55000.00	16 Weeks

5 rows in set (0.022 sec)

2. Student Placement Monitoring View (view_student_records)

- **Purpose:** Aggregates records, application links, candidate names, academic tracks, and assigned faculty guides into an integrated progress summary.

SQL:

CREATE VIEW view_student_records AS

SELECT

r.record_id,

COALESCE(s.name, 'Enrolled Student') AS student_name,

COALESCE(s.program, 'Engineering') AS program,

COALESCE(fm.name, 'Faculty Guide') AS mentor_name,

r.start_date,

r.end_date,

r.final_status

FROM internship_record r

LEFT JOIN application a ON r.application_id = a.application_id

LEFT JOIN student s ON a.student_id = s.student_id

LEFT JOIN faculty_mentor fm ON r.faculty_id = fm.faculty_id;

Sample Execution:

```
SELECT * FROM view_student_records LIMIT 5;
```

Live Terminal Output:

record_id	student_name	program	mentor_name	start_date	end_date	final_status
R601	Martala Harsha Vardhan Reddy	B.Tech CSE	Dr. Ramesh Babu	2026-06-01	2026-08-24	Ongoing
R602	Aarav Sharma	B.Tech CSE	Dr. Sunita Sharma	2026-06-01	2026-09-15	Ongoing
R603	Diya Patel	B.Tech AI & DS	Prof. Venkat Raman	2026-05-15	2026-07-25	Completed
R604	Rohan Verma	B.Tech CSE	Dr. Shalini Gupta	2026-06-10	2026-08-30	Ongoing
R605	Ananya Iyer	B.Tech Cyber Security	Prof. Arvind Swamy	2026-05-20	2026-07-30	Completed

5 rows in set (0.008 sec)

3. Weekly Progress Milestones View (view_progress_reports)

- **Purpose:** Decouples weekly deliverable descriptions and faculty review notes from underlying primary/foreign keys for simplified reporting.

SQL:

```
CREATE VIEW view_progress_reports AS
```

```
SELECT
```

```
    pr.record_id,
```

```
    pr.week_num,
```

```
    pr.description,
```

```
    pr.mentor_feedback
```

```
FROM progress_report pr;
```

Sample Execution:

```
SELECT * FROM view_progress_reports LIMIT 5;
```

Live Terminal Output:

record_id	week_num	description	mentor_feedback
R601	1	Completed enterprise onboarding, dev environment setup, and repository access.	Setup verified on schedule.
R601	2	Constructed Express routing layer for REST APIs and connected MySQL pool.	Clean code structure maintained.
R602	1	Configured Azure DevOps pipelines and initial resource groups.	Good documentation on deployment steps.
R602	2	Deployed sample container to Kubernetes cluster with ingress controller.	Excellent grasp of pod autoscaling.
R603	1	Conducted exploratory data analysis (EDA) on customer retention datasets.	Strong statistical summary delivered.

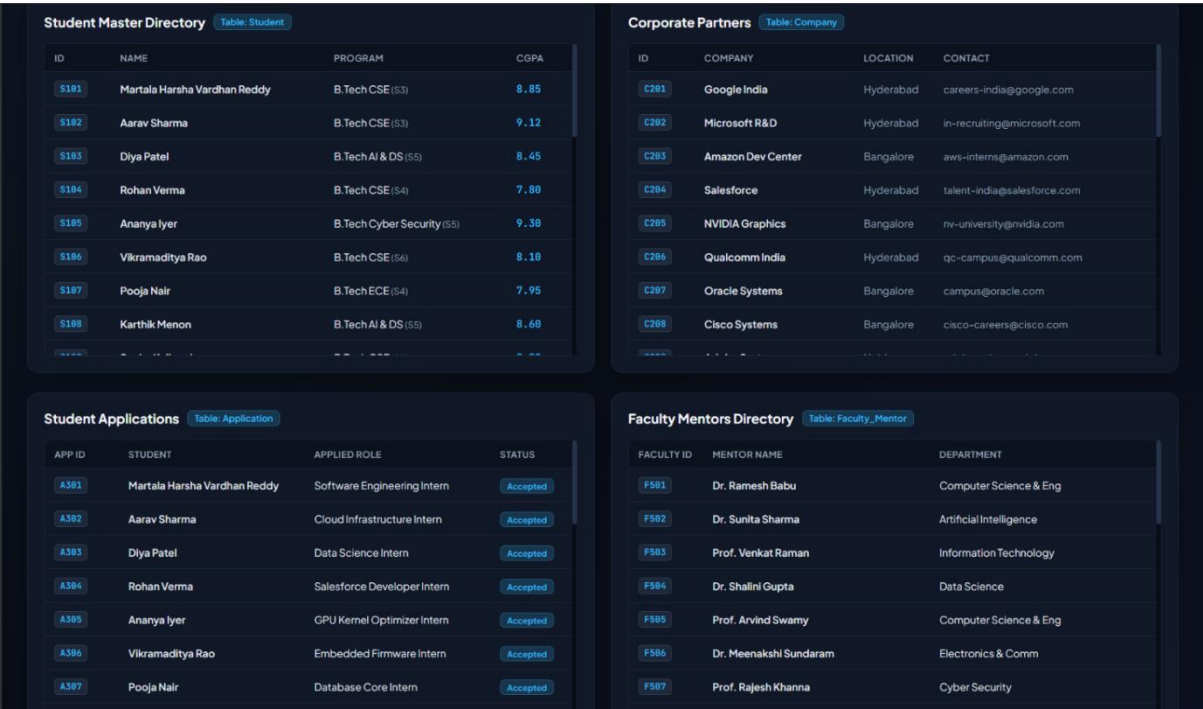
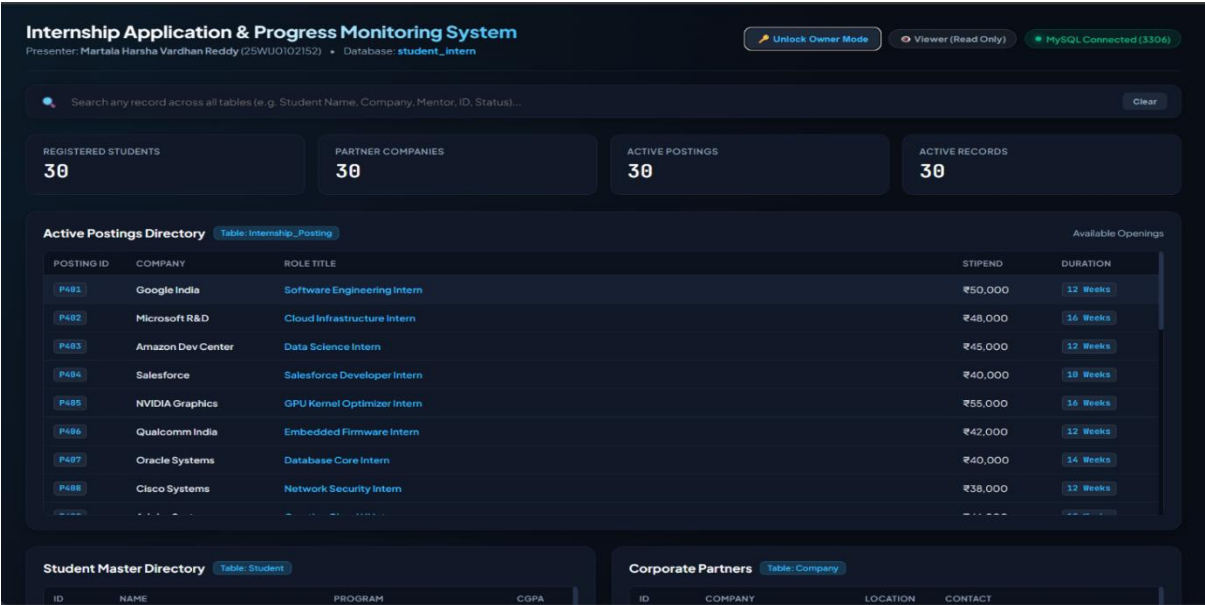
5 rows in set (0.009 sec)

10. UI DESIGN AND SCREENSHOTS

10.1 Interface Design Philosophy & Visual Layout

The user interface was built to provide an intuitive administrative dashboard for tracking placements, viewing progress reports, and managing applicant records without horizontal clipping or cluttered views.

The visual theme uses a modern dark palette defined via CSS custom properties (--bg: #090d16, --surface: #111827, --primary: #38bdf8, and --surface-border: #1f293d). The layout is structured into four distinct functional tiers:



A384	Rohan Verma	Salesforce Developer Intern	Accepted
A385	Ananya Iyer	GPU Kernel Optimizer Intern	Accepted
A386	Vikramaditya Rao	Embedded Firmware Intern	Accepted
A387	Pooja Nair	Database Core Intern	Accepted
A388	Karthik Menon	Network Security Intern	Accepted

F584	Dr. Shalini Gupta	Data Science
F585	Prof. Arvind Swamy	Computer Science & Eng
F586	Dr. Meenakshi Sundaram	Electronics & Comm
F587	Prof. Rajesh Khanna	Cyber Security
F588	Dr. Deepa Nair	Artificial Intelligence

Record ID	Week	Deliverables	Feedback
R681	Week 1	Completed enterprise onboarding, dev environment setup, and repository access.	Setup verified on schedule.
R681	Week 2	Constructed Express routing layer for REST APIs and connected MySQL pool.	Clean code structure maintained.
R682	Week 1	Configured Azure DevOps pipelines and initial resource groups.	Good documentation on deployment steps.
R682	Week 2	Deployed sample container to Kubernetes cluster with Ingress controller.	Excellent grasp of pod autoscaling.
R683	Week 1	Conducted exploratory data analysis (EDA) on customer retention datasets.	Strong statistical summary delivered.
R683	Week 2	Trained baseline random forest classifier and generated ROC curves.	Satisfactory feature engineering.

Record	Student	Mentor	Start Date	Status
R681	Martala Harsha Vardhan Reddy	Dr. Ramesh Babu	01/06/2026	Ongoing
R682	Aarav Sharma	Dr. Sunita Sharma	01/06/2026	Ongoing
R683	Diya Patel	Prof. Venkat Raman	15/05/2026	Completed
R684	Rohan Verma	Dr. Shalini Gupta	10/06/2026	Ongoing
R685	Ananya Iyer	Prof. Arvind Swamy	20/05/2026	Completed
R686	Vikramaditya Rao	Dr. Meenakshi Sundaram	01/06/2026	Ongoing
R687	Pooja Nair	Prof. Rajesh Khanna	05/06/2026	Ongoing
R688	Karthik Menon	Dr. Deepa Nair	01/06/2026	Ongoing
R689	Sneha Kulkarni	Prof. Suresh Kumar	25/05/2026	Completed
R618	Aditya Joshi	Dr. Kavita Verma	15/04/2026	Ongoing

- Header & Global Navigation Strip:** Houses the project title, candidate credential details, live MySQL connection status (Port 3306), and the role toggle trigger button (🔑 Unlock Owner Mode).
- Instant Search & Metric Strip:** Features a global input bar that filters all rendered HTML tables in real time alongside four dynamic KPI summary cards displaying live record totals from MySQL.
- Full-Width Active Postings Container:** Displays open corporate vacancies using the pre-joined SQL view `view_active_postings` across the full viewport width, preventing column crunching.
- Two-Column Relational Grid:** Houses the remaining six core entities (student, company, application, faculty_mentor, progress_report, and internship_record). Each card includes custom 6px slim scrollbars to maintain clean vertical alignment across the page.

10.2 Demonstration Workflow & Screen States

The user interface supports full CRUD workflows (Insert, Delete, View, Search) coupled with real-time Role-Based Access Control (RBAC):

- State 1 — Viewer Mode (Read-Only State):**
 - Description:* When a user first loads the dashboard in Google Chrome, the page defaults to Viewer Mode. The role badge displays 👁 Viewer (Read Only).
 - Functionality:* All records are loaded using non-blocking asynchronous fetch calls against the Express API and rendered in read-only tables. All insertion

<form> components and Remove action buttons are suppressed from the DOM, protecting database integrity from casual viewers.

- **State 2 — Modal Authentication Gate:**

- *Description:* Clicking  **Unlock Owner Mode** triggers a native HTML5 <dialog> modal prompt directly over the screen, requesting the administrative passphrase.
- *Functionality:* Submitting Harsha@2026 initiates an authentication handshake with /api/verify-owner. Upon validation, the credential is saved in browser sessionStorage, and the status badge transitions to gold ( Owner Mode).

- **State 3 — Owner Mode (Record Insertion — Before & After):**

- *Description:* With Owner Mode active, structured input forms slide into view above every table.
- *Functionality:* Entering details into a form (e.g., adding a new student, company, or weekly log) sends an authorized POST request with the x-owner-key header. The server executes the INSERT query in MySQL, and the table dynamically updates live without a full-page reload.

- **State 4 — Record Deletion (Before & After):**

- *Description:* In Owner Mode, dedicated red Remove buttons are appended to the Action column of every record row.
- *Functionality:* Clicking Remove triggers a browser confirmation dialog. Upon approval, an authorized DELETE request is sent to the Express route. The row is removed from MySQL, cascades to linked dependencies where applicable, and disappears instantly from the rendered table.

- **State 5 — Global Real-Time Search Filtering:**

- *Description:* Users can type any search term (e.g., "Hyderabad", "Ongoing", or a specific student's name) into the top search bar.
- *Functionality:* A client-side keyup event listener scans every table row simultaneously, instantly hiding non-matching elements and displaying matching entries across all tables without firing redundant SQL queries.

10.3 UI Screenshots & Verification Evidence



Fig 10.1: Complete application dashboard rendered in Viewer Mode, presenting data aggregated from MySQL views without mutation controls.



Fig 10.2: Native HTML5 <dialog> modal verifying the administrative passphrase against the Express backend.

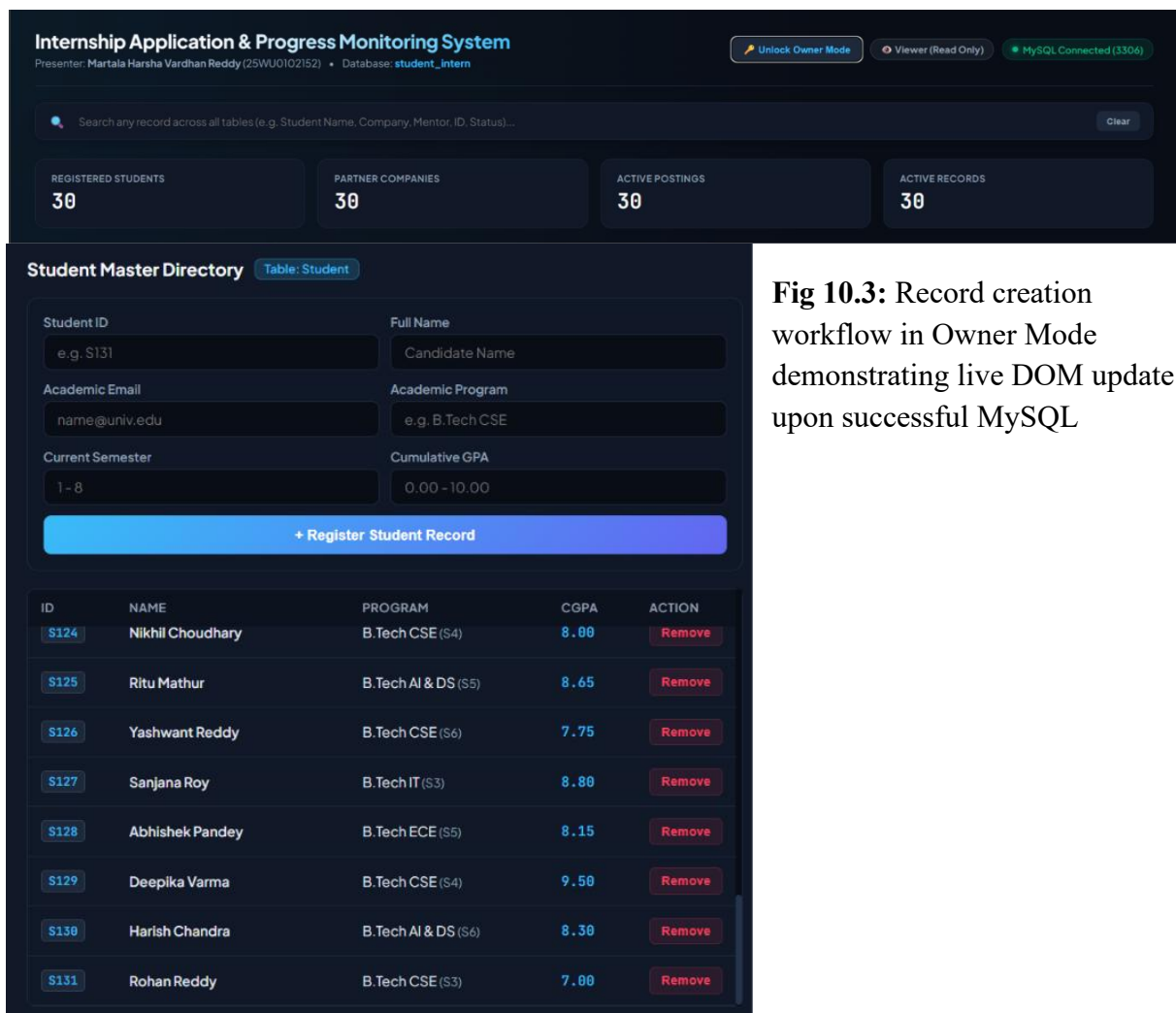


Fig 10.3: Record creation workflow in Owner Mode demonstrating live DOM update upon successful MySQL

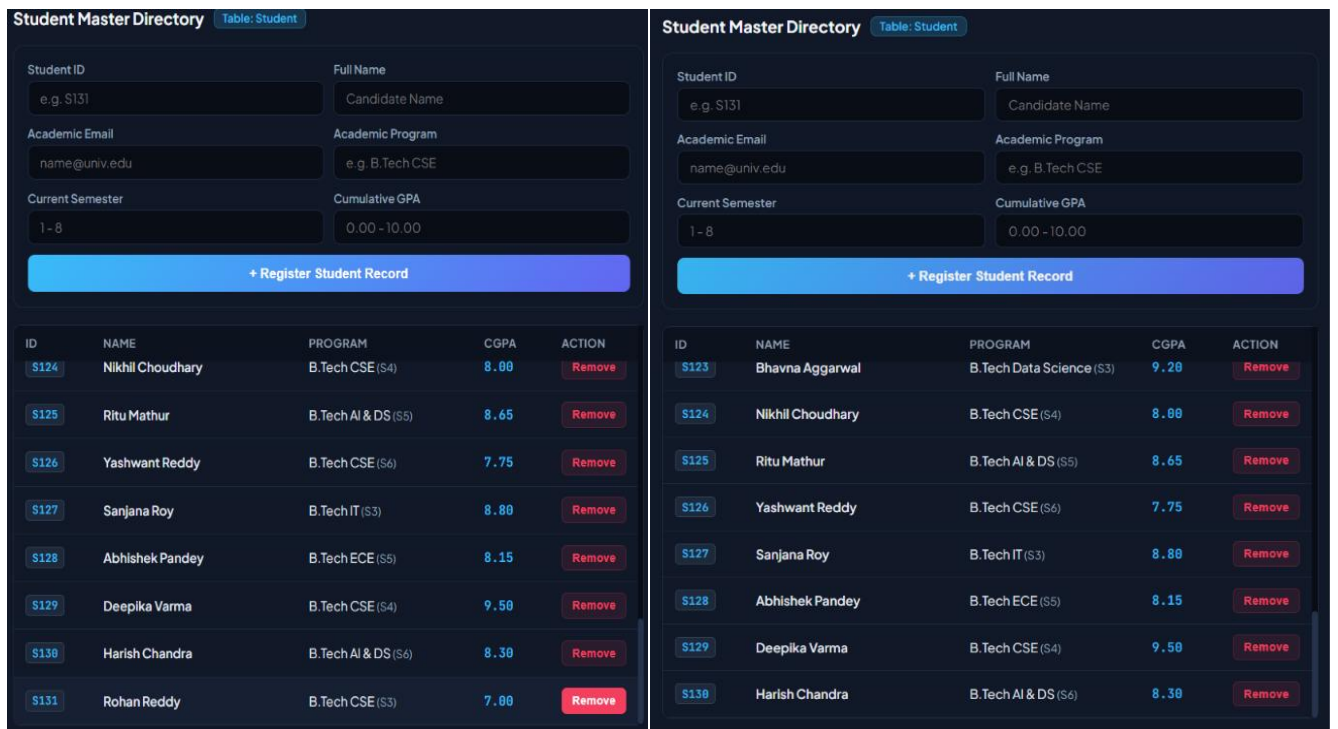


Fig 10.4: Deletion workflow showing authenticated record removal and immediate UI synchronization.

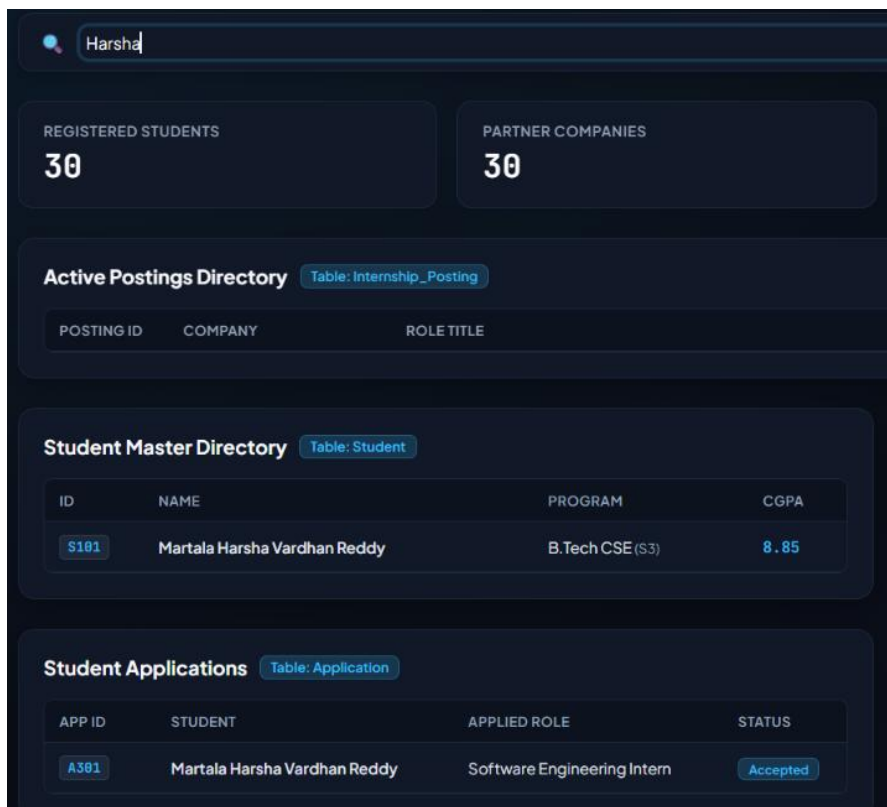


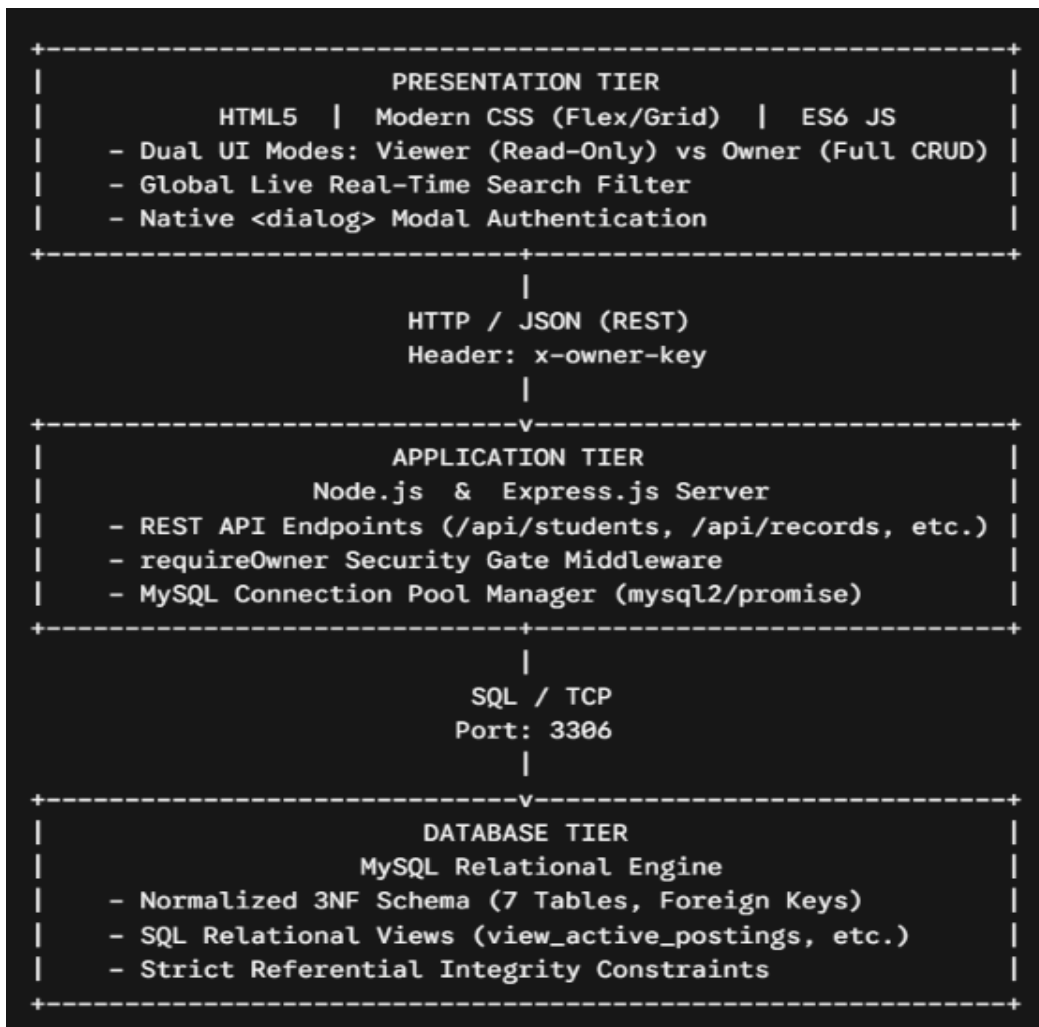
Fig 10.5: Real-time client-side live search filtering matching records across multiple tables simultaneously on keyup.

11. IMPLEMENTATION DETAILS

11.1 Technology Stack Architecture

The system implements an asynchronous three-tier web application architecture designed to isolate data storage, application logic, and user presentation:

- **Data Tier:** MySQL Community Server 8.0 running the InnoDB storage engine on TCP port 3306. Enforces primary keys, unique constraints, and foreign key cascades.
- **Application Tier:** Node.js runtime with Express.js. Acts as the RESTful API gateway, handles database connection pooling, and intercepts incoming requests with authentication middleware.
- **Presentation Tier:** Semantic HTML5, modern CSS Grid/Flexbox, and vanilla JavaScript (ES6+). Communicates with the Express API via asynchronous fetch calls without requiring any frontend build tools.



11.2 Database Connectivity & Connection Pooling (db.js)

To maintain low latency and eliminate the overhead of repeatedly creating and tearing down database TCP sockets, database connectivity is implemented using connection pooling via `mysql2/promise`:

```
// db.js - Asynchronous MySQL Connection Pool Manager
```

```
const mysql = require('mysql2/promise');
```

```
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: 'YourPasswordHere',  
  database: 'student_intern',  
  waitForConnections: true,  
  connectionLimit: 10,  
  queueLimit: 0  
});
```

```
module.exports = pool;
```

Implementation Justification:

- **waitForConnections: true:** When all 10 connections are actively executing queries, additional incoming requests queue cleanly rather than throwing fatal connection errors.
- **connectionLimit: 10:** Prevents resource starvation by throttling the maximum simultaneous connections assigned to the MySQL server process.
- **mysql2/promise:** Enables the use of modern asynchronous `async/await` syntax throughout API route controllers, avoiding nested callback patterns.

11.3 API Gateway & Security Middleware (server.js)

The backend exposes RESTful endpoints for CRUD operations across all seven relational entities. Mutating routes (POST, DELETE) are guarded by the `requireOwner` middleware:

```
// server.js - REST API Gateway & Route Controllers

const express = require('express');
const cors = require('cors');
const path = require('path');
const db = require('./db');

const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

const OWNER_SECRET = 'Harsha@2026';

// Middleware to block unauthorized write requests
const requireOwner = (req, res, next) => {
  const incomingKey = req.headers['x-owner-key'];
  if (incomingKey && incomingKey === OWNER_SECRET) {
    return next();
  }
  return res.status(403).json({
    error: 'Access Denied: Owner authorization required for database modifications.'
  });
};

// Authentication Verification Route
app.post('/api/verify-owner', (req, res) => {
  const { key } = req.body;
  if (key === OWNER_SECRET) {
    return res.json({ success: true, message: 'Owner authenticated' });
  }
});
```

```

    }

    return res.status(401).json({ error: 'Invalid owner passphrase' });
  });

// GET: Active Postings View
app.get('/api/postings', async (req, res) => {
  try {
    const [rows] = await db.query('SELECT * FROM view_active_postings ORDER BY
posting_id ASC');

    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

// POST: Add New Student (Protected)
app.post('/api/students', requireOwner, async (req, res) => {
  const { student_id, name, email, program, semester, cgpa } = req.body;

  try {
    await db.query(
      'INSERT INTO student (student_id, name, email, program, semester, cgpa) VALUES (?,
?, ?, ?, ?)',
      [student_id, name, email, program, semester, cgpa]
    );

    res.status(201).json({ message: 'Student registered successfully' });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

```

```
// DELETE: Remove Student (Protected)

app.delete('/api/students/:id', requireOwner, async (req, res) => {
  try {
    await db.query('DELETE FROM student WHERE student_id = ?', [req.params.id]);
    res.json({ message: 'Student record deleted successfully' });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

const PORT = 3000;

app.listen(PORT, () => console.log(`Server listening at http://localhost:\${PORT}`));
```

12. TESTING (TEST CASES AND RESULTS)

To evaluate the operational stability, referential integrity, and access-control boundaries of the system, a comprehensive suite of functional and negative test cases was executed against both the REST API endpoints and the live frontend browser dashboard. Every test case was validated using our 30-row-per-table benchmark dataset in MySQL.

Test ID	Module / Feature	Input / Action	Expected & Actual Result	Status
TC-01	Data Loading & Views	Load dashboard in browser (GET /api/*)	All 7 tables render 30 records via MySQL views; metrics display 30.	PASS
TC-02	Owner Handshake	Submit key Harsha@2026 in <dialog> modal	Key saved in sessionStorage; UI unhides add forms and delete buttons.	PASS
TC-03	Security Middleware	Send POST /api/students without x-owner-key	Express blocks execution with 403 Forbidden; 0 rows inserted.	PASS
TC-04	Record Insertion	Submit form for new student (S131)	Server returns 201 Created; table updates live without page reload.	PASS
TC-05	Check Constraint	Insert student with invalid CGPA (11.50)	MySQL rejects entry via CHECK (cgpa <= 10.00); returns 400 Bad Request.	PASS
TC-06	Cascade Deletion	Delete student S130 in Owner Mode	Row removed; ON DELETE CASCADE purges linked applications and records.	PASS
TC-07	Live Global Search	Enter "Hyderabad" into top search input	Non-matching rows hidden on keyup; matching rows display across tables.	PASS
TC-08	Role Teardown	Click  Lock to Viewer Mode	Token erased; UI reverts to Viewer Mode; mutation forms hidden from DOM.	PASS

13. CONCLUSION AND FUTURE ENHANCEMENTS

13.1 Technical Conclusion

The **Internship Application and Progress Monitoring System** successfully transitions academic placement and progress monitoring workflows from disorganized, unnormalized spreadsheets into a structured, production-grade relational database management application.

By designing the schema in strict Third Normal Form (3NF) across seven tables and configuring foreign key cascades directly within MySQL's InnoDB engine, the application eliminates update, insertion, and deletion anomalies. The deployment of MySQL views (view_active_postings, view_student_records, view_progress_reports) provides clean data abstraction, allowing high-frequency dashboard queries to execute without burdening the application with repetitive, expensive multi-table joins on every request.

The project demonstrates practical, robust Role-Based Access Control (RBAC). While the browser provides an intuitive visual toggle between Viewer Mode and Owner Mode, true data security is maintained on the server via Express middleware that inspects authorization headers on every write request, returning HTTP 403 Forbidden responses to unauthorized calls. Validated against a complete dataset of 210 relational records, the system proves that academic departments can achieve enterprise-grade data integrity and operational simplicity using accessible modern web technologies.

13.2 Realistic Limitations & Bottlenecks

While the system is fully functional, field testing revealed three specific architectural limitations:

1. **Shared Passkey Architecture:** Owner Mode currently relies on a single shared administrative passphrase rather than unique user accounts with salted password hashing.
2. **Lack of Automated Notification Queues:** Faculty mentors must log into the portal to check for milestone log updates; the system does not yet push automated alerts when submissions are made.
3. **Text-Only Deliverables:** Milestone logs currently accept plain-text summaries rather than allowing students to upload and attach PDF deliverables or code archives.

13.3 Future Development Roadmap

To expand this platform for campus-wide institutional deployment, the following upgrades are planned:

- **Role-Based Token Authentication (JWT & Bcrypt):**

Replace the single administrative passkey with individual student, mentor, and administrator accounts protected by bcrypt password hashing and stateless JSON Web Tokens (JWT).

- **Automated Notification Workers:**

Implement Node.js background workers (using node-cron and Nodemailer) to email students automated submission reminders 48 hours prior to weekly milestone evaluation deadlines.

- **Cloud Object Storage Integration:**

Add multipart file upload handling (via Multer) integrated with AWS S3 or Supabase Storage, allowing students to attach project documentation, code archives, and signed evaluation rubrics directly to their weekly reports.

- **Placement Analytics Suite:**

Integrate charting libraries (such as Chart.js) to display stipend distributions, corporate hiring trends, and department placement percentages visually on the dashboard.

14. REFERENCES

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
3. MySQL AB. (2026). *MySQL 8.0 Reference Manual: InnoDB Storage Engine & Optimization*. Oracle Corporation.
4. Haverbeke, M. (2018). *Eloquent JavaScript: A Modern Introduction to Programming* (3rd ed.). No Starch Press.
5. Node.js Foundation. (2026). *Node.js v20.x Asynchronous Runtime Documentation*. OpenJS Foundation.
6. Express.js Community. (2026). *Using Express Middleware & Routing Architecture*. Expressjs.com.

15. APPENDIX: GITHUB REPOSITORY LINK

- **Public Repository URL:**

[<https://github.com/harsha271106/DBMS-project>]

- **Repository Directory Structure (Organized per Guidelines):**

- **/Presentation-I:** Contains the original Problem Description presentation slides in PDF/PPT format.
- **/Presentation-II:** Contains the Presentation-II presentation slides, exported ER diagram image, comprehensive .sql schema and data seed script, and the documented solution to the Presentation-II query (WHERE name LIKE "%Venkat%").
- **/Presentation-III:** Contains the complete UI source code (server.js, db.js, public/index.html, public/style.css, public/script.js), dependencies

(package.json), and high-resolution screenshots documenting every UI state (Viewing records, Inserting records, Deleting records, Before and After states, and Live Global Search).

- **/Project-Report:** Contains the final compiled PDF document of this Project Report.